```
htmlObj = BeautifulSoup(r.content, 'html.parser')
print(htmlObj.prettify())
```

The output is:

```
200
<!DOCTYPE html>
<html lang="en">
<head>
<!-- Required meta tags always come first -->
<meta charset="utf-8"/>
<meta content="width=device-width, initial-scale=1,
shrink-to-fit=no" name="viewport"/>
<meta content="ie=edge" http-equiv="x-ua-compatible"/>
<title>

Flask Bulma CSS - BulmaPlay Open-Source App | AppSeed App
Generator
```

For further analysis, it's possible to item-wise parse the HTML page into different parent and child classes.

```
import requests
from bs4 import BeautifulSoup
# Making a GET request
r = requests.get('https://flask-bulma-css.appseed.us/')

# Parsing the HTML
htmlobj = BeautifulSoup(r.content, 'html.parser')

# Getting the title tag
print(htmlobj.title)

# Getting the name of the tag
print(htmlobj.title.name)

# Getting the name of parent tag
print(htmlobj.title.parent.name)
# use the child attribute to get
# The name of the child tag
print(htmlobj.title.children.__class__)
```

The output is:

```
<title>Python Tutorial | Learn Python Programming</title>
title
head
<class 'list_iterator'>
```

Finding HTML page elements

Now, we want to extract some valuable information from the HTML text. The retrieved content in the *htmlobj* object may contain some nested structures. From these nested structures, different HTML elements can be parsed to retrieve valuable information from the web page.

HTML element search by class

We can see in Figure 4 that there is a *div* tag with the *class* tag entry. The *found* class method can be used to locate the tags. This

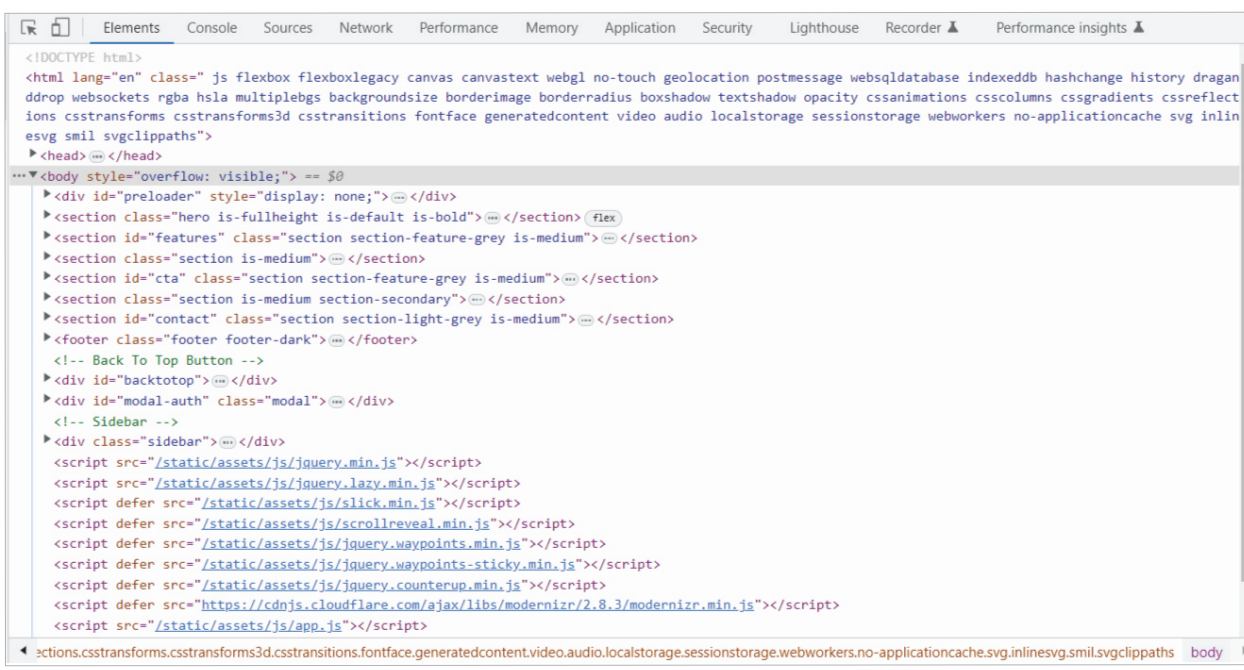

Figure 4: Inspection of web page script using a browser's developer tool